Web Penetration Testing Project

OUT TEAM:

- Mark Mekhael Adly
- Hassan Amgad Hassan
- Shahd Ahmed Mohamad
- Mohamad Mahmoud Ali

Scope : (demo.owasp-juice.shop) & (preview.owasp-juice.shop):

▼ 1. Executive Overview

The objective of this penetration test was to evaluate the security of the Juice Shop application. The assessment aimed to identify weaknesses within the web platform, including user-facing components and backend infrastructure. This engagement focused on simulating real-world attack scenarios to measure the application's resilience against common threats.

- **Tested Application**: Juice Shop
- **Testing Mode**: Black-box
- **Assessment Focus**: Web application-level vulnerabilities
- **Approach**: Aligned with OWASP Top 10 and industry best practices

▼ Summary of Findings:

- **CSRF (Cross-Site Request Forgery)**

Attacker can trick authenticated users into executing unwanted actions.

- **Broken Access Control**

  Unauthenticated users can access restricted endpoints.

- **IDOR (Insecure Direct Object Reference)**

  Users can access others' data by manipulating object IDs.

- **Broken Authentication via Predictable OAuth State**

  Attacker can guess state values to hijack sessions.

- **SQL Injection in the Login Page**

  Input fields are vulnerable to SQL payloads for login bypass.

- **Authentication Bypass via SQL Injection**

  Malicious queries can log in without valid credentials.

- **JWT Token Disclosure**

  Admin credentials can be extracted from tokens.

- **SQLi to Bypass Login for Another User**

  Login process fails to validate user identity securely.

- **Login via SQLi With Non-Existent Account**

  Database logic allows fake users to gain access.

- **Backup File Discovery (Gobuster)**

  Exposed files reveal sensitive data or source code.

- **Weak Encryption in Coupons**

  Encryption flaws allow decoding and manipulation of discount codes.

- **Cookies Missing HttpOnly Flag**

  Client-side scripts can access sensitive cookie data.

▼ Key Security Recommendations:

- Implement anti-CSRF tokens

Validate request origin and method on state-changing operations.

- Fix access control logic

  Enforce authentication and role-based authorization on all sensitive actions.

- Protect object references

  Validate user authorization before accessing internal IDs or resources.

- Secure OAuth flows

  Use unpredictable, unique state values. Validate them on callback.

- Prevent SQL Injection

  Use parameterized queries. Avoid dynamic SQL generation.

- Secure JWT tokens

  Store them securely. Set short expiry. Avoid exposing sensitive data in tokens.

- Remove exposed backup files

  Block access to unused or temporary files. Use proper file permissions.

- Use strong encryption

  Secure coupon logic with validated algorithms and proper key management.

- Set HttpOnly on cookies

  Prevent access to session tokens via JavaScript.

▼ General risk level:

- **CSRF** — High

  Allows unauthorized actions on behalf of authenticated users. Can lead to account changes or data tampering.

- **Broken Access Control** — Critical

  Exposes restricted areas or functions. Enables unauthorized data access or privilege escalation.

- **IDOR** — High

  Lets attackers access or manipulate data by changing object IDs. Violates data confidentiality.

- **Broken Authentication via Predictable OAuth** — Critical

  Allows session hijacking. Attackers can impersonate users or gain unauthorized access.

- **SQL Injection (All types)** — Critical

  Direct access to database. Can bypass logins, extract data, or create unauthorized accounts.

- **JWT Token Disclosure** — High

  Gives attackers valid credentials. Leads to full session compromise.

- **Backup File Discovery** — Medium

  Reveals internal source code or config files. Can expose further vulnerabilities.

- **Weak Encryption in Coupons** — Medium

  Allows tampering with discount logic. Can be used to gain unauthorized benefits.

- **Cookies Missing HttpOnly Flag** — Medium

  Exposes session tokens to JavaScript. Enables session theft via XSS.

▼ Key Security Recommendations:

- Enforce anti-CSRF tokens and validate request origins.
- Sanitize and encode all user input and output. Apply CSP headers.
- Disable external entity processing in XML parsers.
- Use parameterized queries to prevent SQLi.
- Apply RBAC and ensure objects are protected by access controls.
- Whitelist allowed URLs and block internal endpoints to mitigate SSRF.
- Sanitize inputs used in dynamic code execution.
- Validate redirect destinations before processing.

- Restrict file path inputs and validate inclusion targets.
- Close unused ports, implement 2FA, and remove default credentials.

General recommendations also include routine vulnerability assessments, WAF deployment, real-time activity monitoring, and patch management.

▼ 2. Testing Strategy

A methodical process based on OWASP Top 10 2023 and modern attack methodologies guided the penetration test. Both automated tools and manual analysis were applied.

Scope & Method:

- **Scope**: Comprehensive web application assessment including backend components
- **Methodology**: Risk-based and OWASP-aligned testing

Tools Utilized:

- Vulnerability Scanning: `Acunetix` , `Nuclei`
- **Discovery Tools**: `subfinder` for subdomain enumeration
- **Directory Brute Forcing**: `gobuster`
- **Wordlists**: `SecLists`
- **Vulnerability Detection**: `Burp Suite Pro`
- **Exploitation Tools**: Custom scripts, SQLmap
- **Manual Review Areas**: CSRF, session handling, authentication, business logic, and access control

▼ Testing Phases:

- **Reconnaissance:**
  - Enumerated subdomains and directories to map the structure.

    - Vulnerability Scanning
    - Identified an open admin interface on port 8000.
- **Scanning:**
    - Used automated and manual tools to identify issues like XSS, SQLi, IDOR, and authentication flaws.
- **Exploitation:**
    - Demonstrated data extraction via SQLi.
    - Performed JavaScript injection to show session hijacking.
    - Proved CSRF attacks could be executed to manipulate user actions.
    - Accessed admin functions using default credentials.
- **Post-Exploitation:**
    - Extracted sensitive information including user credentials.
    - Hijacked sessions and demonstrated account takeover.
    - Used administrative privileges to manipulate backend systems.
- **Validation & Reporting:**
    - Each vulnerability was validated manually.
    - Provided visual evidence and reproduction steps.
    - Tailored remediation steps were proposed for each issue.

---

1. Reconnaissance

Target: demo.owasp-juice.shop

1. **Introduction**

   This report summarizes the reconnaissance findings for the **OWASP Juice Shop** (demo.owasp-juice.shop), an intentionally vulnerable web application designed for security training. The goal was to gather information about the target's infrastructure, subdomains, SSL/TLS configurations, and exposed services.

2. **Methodology**

   The reconnaissance phase included:

   - **Subdomain Enumeration**
   - **SSL/TLS Analysis**
   - **Directory and File Enumeration**
   - **DNS and MX Record Analysis**

   Tools used:

   - **Nuclei** (for SSL/TLS and DNS checks)
   - **Gobuster** (for directory brute-forcing)
   - **Manual DNS queries**

3. **Findings**

   3.1 Subdomains and Hosts

   Multiple subdomains were identified under owasp-juice.shop:

Subdomain	Purpose (Inferred)
demo.owasp-juice.shop	Main demo instance
preview.owasp-juice.shop	Preview/testing environment
stats.owasp-juice.shop	Analytics/metrics
pwning.owasp-juice.shop	Security training
intro.owasp-juice.shop	Introduction page
help.owasp-juice.shop	Support/help page
slides.owasp-juice.shop	Presentation slides

   Observation:

   - Wildcard TLS certificate in use (*.owasp-juice.shop).

   3.2 SSL/TLS Configuration

   - **Issuer:** DigiCert Inc.
   - **Supported Protocols:** TLS 1.2 and TLS 1.3 (modern configurations).
   - **Wildcard Certificate:** Present (*.owasp-juice.shop).
   - **SANs (Subject Alternative Names):**

- .owasp-juice.shop
- owasp-juice.shop

Vulnerabilities:

- No outdated protocols (SSLv3, TLS 1.0/1.1) detected.
- **Recommendation:** Ensure HSTS (HTTP Strict Transport Security) is enforced.

3.3 DNS and MX Records

- **MX Records:** All subdomains point to smtpin.rzone.de (likely a mail relay).
- **CAA Records:** Present, indicating Certificate Authority Authorization (restricts which CAs can issue certs).

Implications:

- No direct email server exposure; relies on a third-party mail service.
- Proper CAA setup reduces risk of rogue certificate issuance.

3.4 Directory and File Enumeration

Gobuster scan results (partial):

Path	Status	Notes
/assets	301	Static resources
/ftp	200	Exposed FTP-like interface
/metrics	200	Prometheus metrics (unauthenticated)
/api	500	Internal server error
/rest/products/search	500	SQL injection vector (found in later testing)
/Profile	500	Possible improper access control
/snippets	200	

Key Findings:

- /metrics exposes system/application metrics (potential info leak).
- /ftp may allow file upload/download (further testing needed).
- Multiple 500 errors suggest debugging mode or misconfigurations.

3.5 Additional Observations

- **Reverse Proxy Detected:** The server uses a reverse proxy (likely Nginx/Cloudflare).
- **No CSP (Content Security Policy):** Missing security headers increase XSS risks.
- **Open Ports (Inferred):**
  - 443 (HTTPS) – Active
  - 80 (HTTP) – Likely redirects to HTTPS

4. **Summary of Risks**

Risk	Severity	Details
Exposed Prometheus Metrics	Medium	Unauthenticated access to /metrics
Wildcard TLS Certificate	Low	Risk if private key is compromised
Missing Security Headers	Low-Medium	No CSP, HSTS, or X-Frame-Options
Development Subdomains	Low	May expose test environments

1. **Recommendations**
2. **Restrict Access to** /metrics**:**
   - Add authentication or IP whitelisting.
3. **Implement Security Headers:**
   - Content-Security-Policy, X-Frame-Options: DENY, Strict-Transport-Security.
4. **Monitor Subdomains:**
   - Remove or restrict access to development/testing subdomains (local*).
5. **Review MX Records:**
   - Ensure no misconfigurations in email handling.

5. **Conclusion**

   The reconnaissance phase revealed a well-configured TLS setup but exposed several areas for improvement, including:

- Unprotected system metrics (/metrics).
- Lack of security headers (CSP, HSTS).
- Development subdomains that could be targeted.

Next Steps: Proceed with vulnerability scanning and penetration testing, focusing on:

- SQL injection (/rest/products/search).
- Exposed FTP functionality (/ftp).
- XSS and header-related vulnerabilities.

20250506_Comprehensive_new__http_demo_owasp_juice_shop.html

20250506_Comprehensive_new__http_demo_owasp_juice_shop.pdf

Juice-Shop-Nuclei.txt

Juice_Shop_gobuster.txt

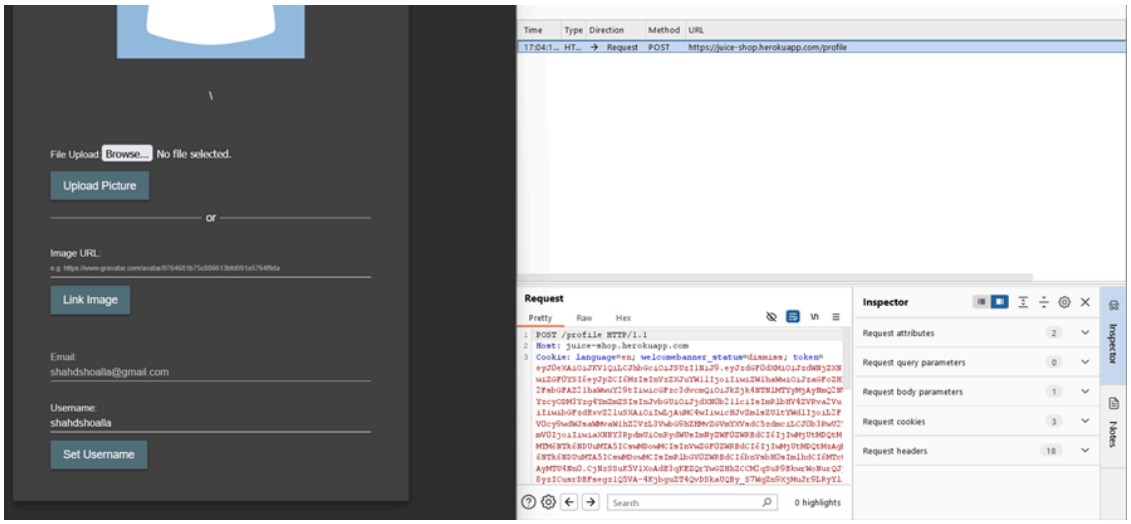
---

2. Scan & Exploit

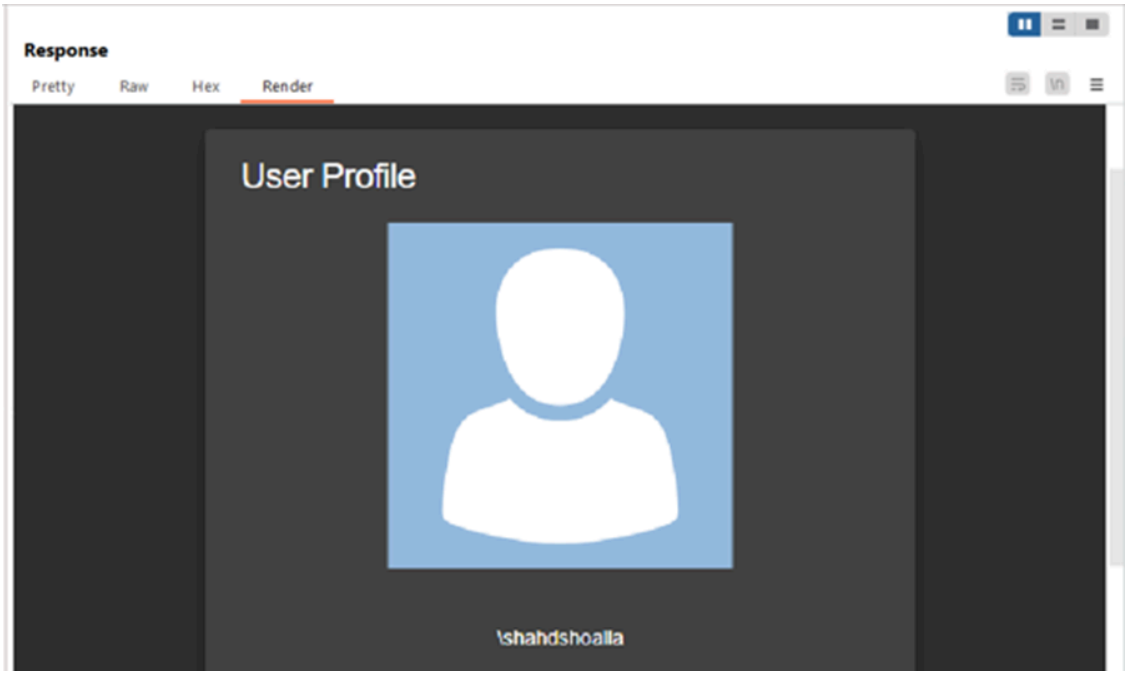
2.1 Cross-Site Request Forgery (CSRF)

- Severity : Medium
- **Step 1: Send Profile Update Request**

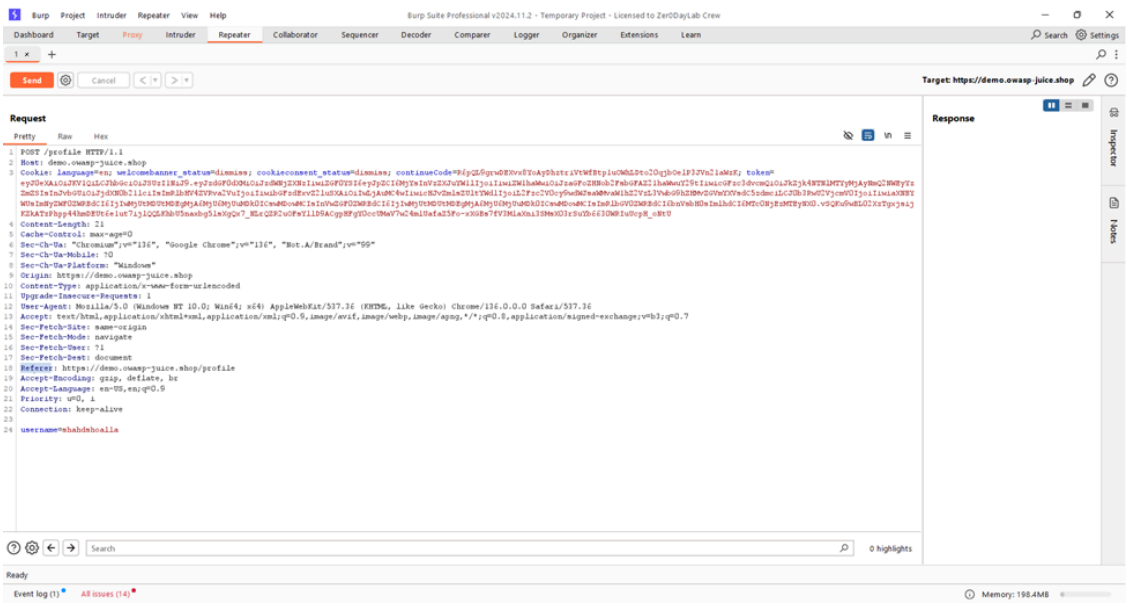
  - Action: Changed the username from the profile settings page
  - Tool: Captured the request in Burp Suite (Proxy > HTTP history)
  - Endpoint: PUT /api/Users/profile
  - Headers included a Referer pointing to the Juice Shop origin



- **Step 2: Send Request to Repeater**
  - Sent the same request to Burp Suite Repeater
  - Confirmed the server accepted the request and changed the username
  - This shows the server accepts profile updates if the user is authenticated

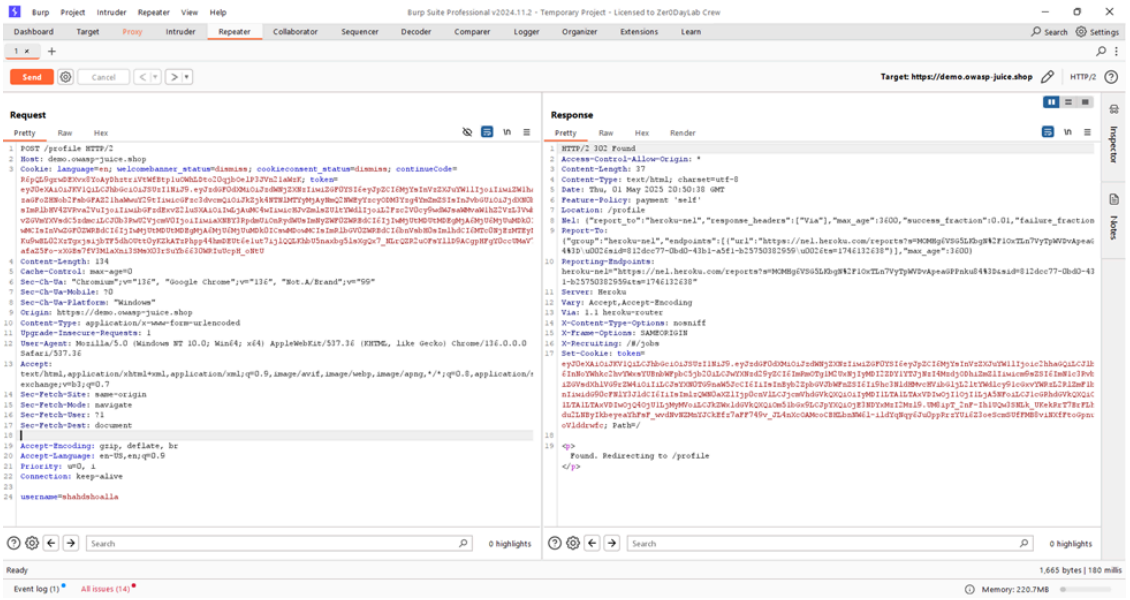


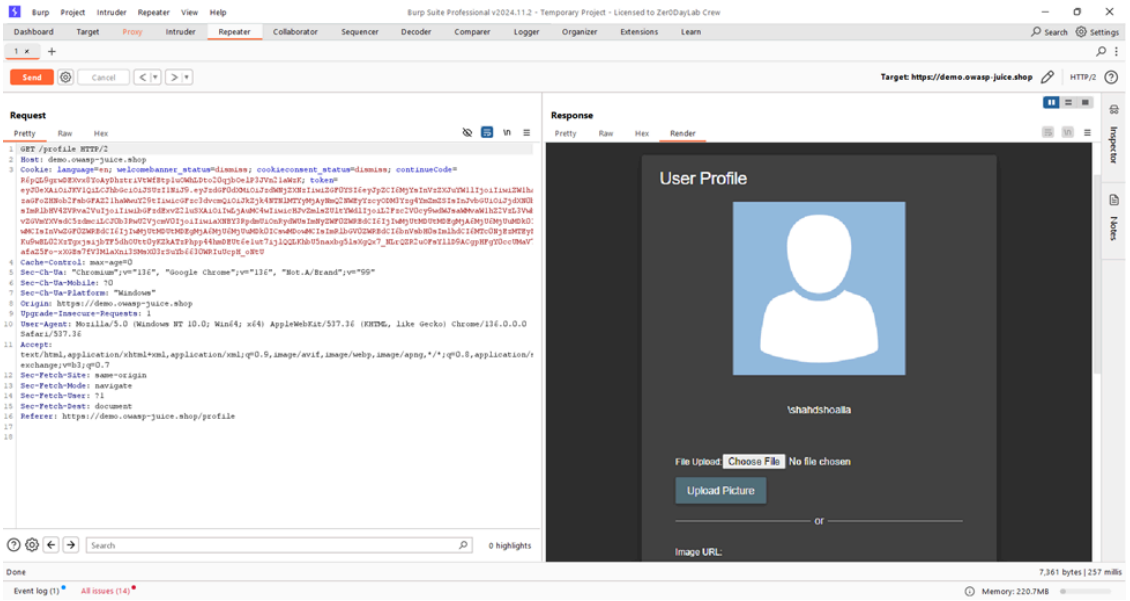
- **Step 3: Identify Referer Header**
  - Observed the Referer header in the request
  - Value: https://demo.owasp-juice.shop/account/profile



- **Step 4: Remove Referer Header**
  - Deleted the Referer from the request

- Sent the request again
- Result: Server still accepted the request



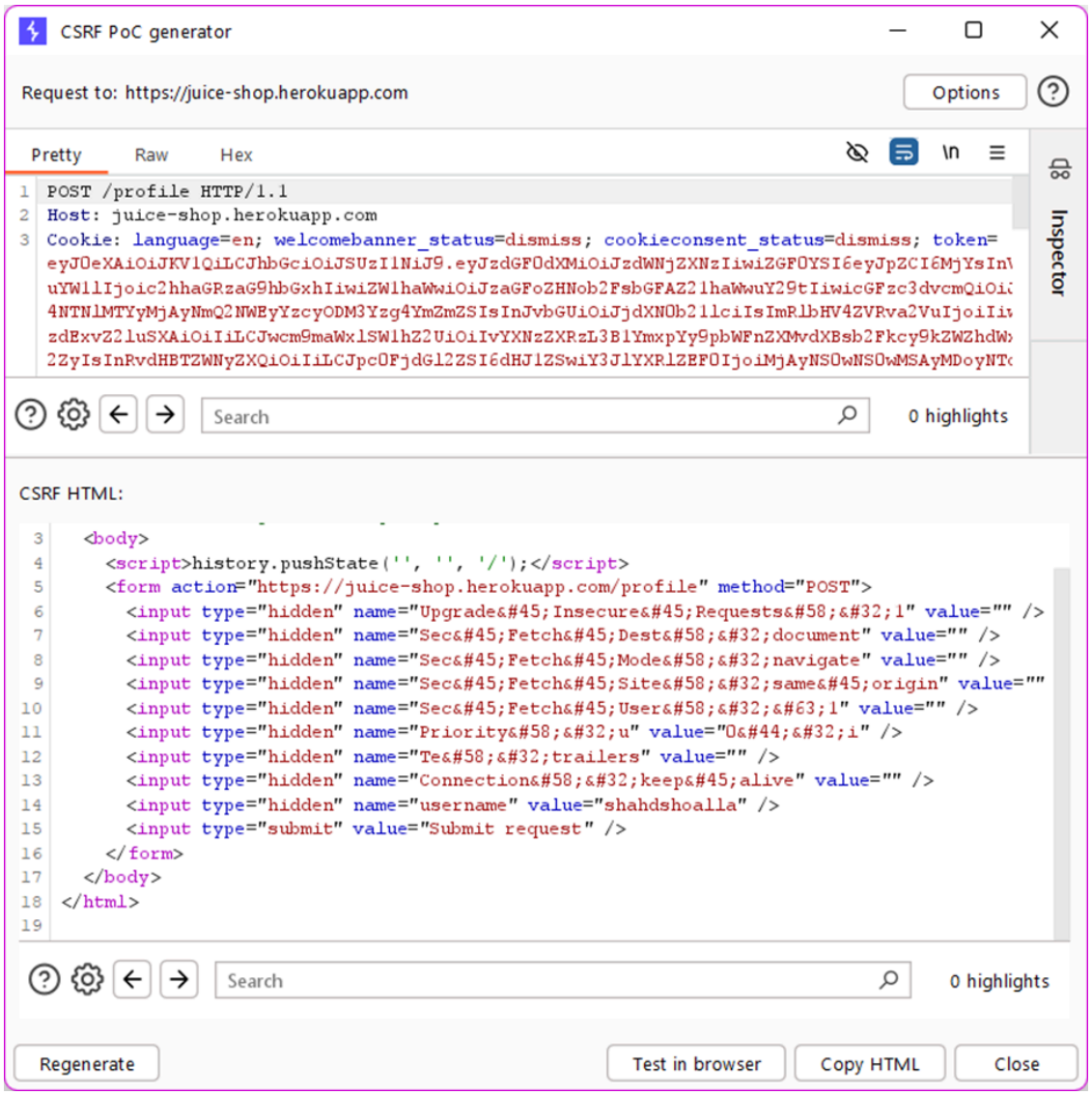


Why This Is a Vulnerability

- A secure server should verify requests come from trusted origins using headers like Referer or Origin
- The absence of these headers should result in the request being rejected

- The application accepted state-changing requests without validating the origin
- This allows attackers to craft a malicious page that submits a request from a different site without the victim's knowledge

- **Step 5: Create CSRF Exploit Form**
  - Used Burp Suite > Engagement Tools > Generate CSRF PoC
  - Got an auto-generated HTML form



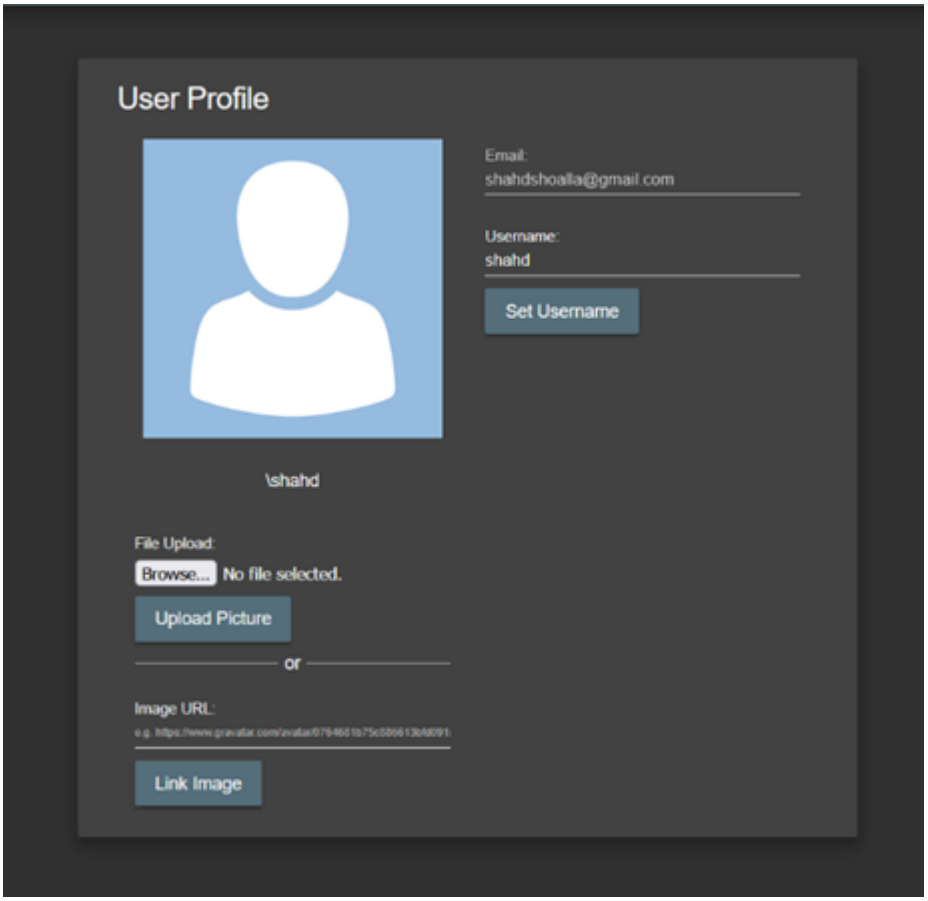
- **Step 6: Edit and Execute the Exploit**
  - Saved the form as a .html file

• Opened the file in a browser
• While logged into Juice Shop, loading this page submitted the form automatically

juiceshop.html

File Edit View

```
<html>
  <!-- CSRF PoC - generated by Burp Suite Professional -->
  <body>
    <script>history.pushState('', '', '/');</script>
    <form action="https://juice-shop.herokuapp.com/profile" method="POST">
      <input type="hidden" name="Upgrade&#45;Insecure&#45;Requests&#58;&#32;1" value="" />
      <input type="hidden" name="Sec&#45;Fetch&#45;Dest&#58;&#32;document" value="" />
      <input type="hidden" name="Sec&#45;Fetch&#45;Mode&#58;&#32;navigate" value="" />
      <input type="hidden" name="Sec&#45;Fetch&#45;Site&#58;&#32;same&#45;origin" value="" />
      <input type="hidden" name="Sec&#45;Fetch&#45;User&#58;&#32;&#63;1" value="" />
      <input type="hidden" name="Priority&#58;&#32;u" value="0&#44;&#32;i" />
      <input type="hidden" name="Te&#58;&#32;trailers" value="" />
      <input type="hidden" name="Connection&#58;&#32;keep&#45;alive" value="" />
      <input type="hidden" name="username" value="shahd" />
      <input type="submit" value="Submit request" />
    </form>
  </body>
</html>
```



Impact

- Attackers can trick logged-in users into submitting unauthorized requests
- This can lead to:
  - Profile tampering
  - Email or password changes
  - Session manipulation
- All without the user knowing or consenting

Recommendations

1. **Require CSRF Tokens**

- Generate and validate a CSRF token for every sensitive form or state-changing request
- Tokens must be user-specific and unpredictable

1. **Enforce Origin or Referer Checks**

- Reject requests that don’t come from your domain
- Validate Origin or Referer headers to ensure requests originate from a trusted source

1. **Use SameSite Cookies**

- Set cookies with SameSite=Strict or Lax to prevent them from being sent in cross-site requests

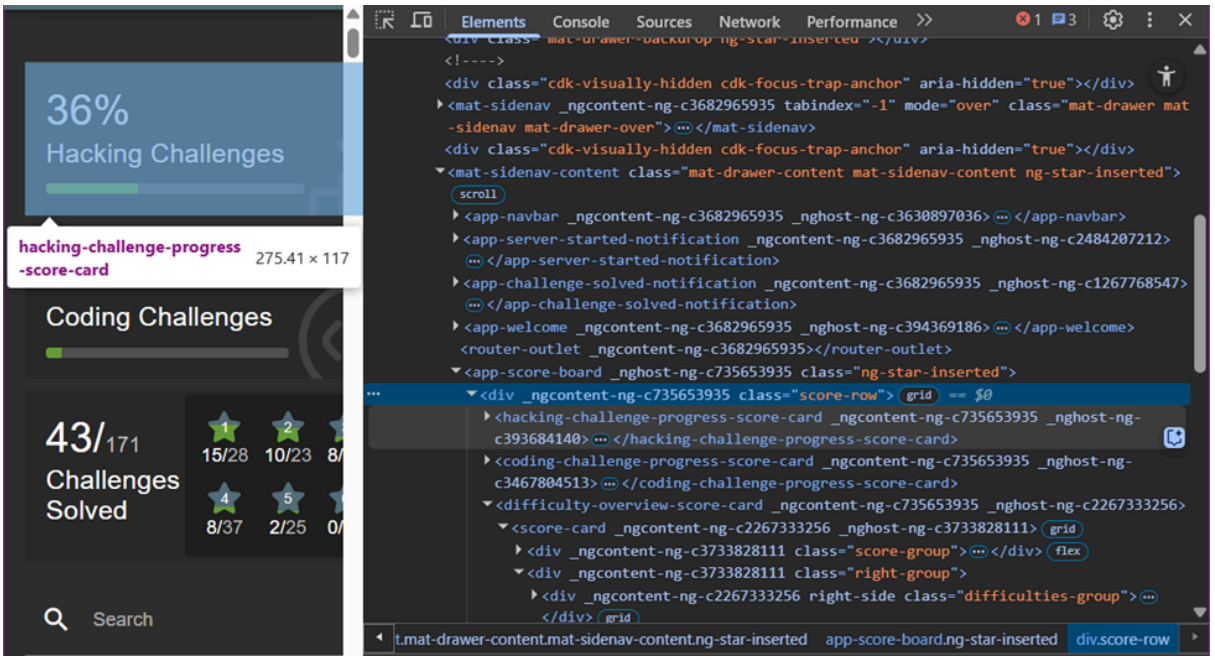
1. **Avoid Using GET for Sensitive Actions**

- Ensure that only POST, PUT, and DELETE methods are used for changing data
- Do not allow sensitive changes via GET requests

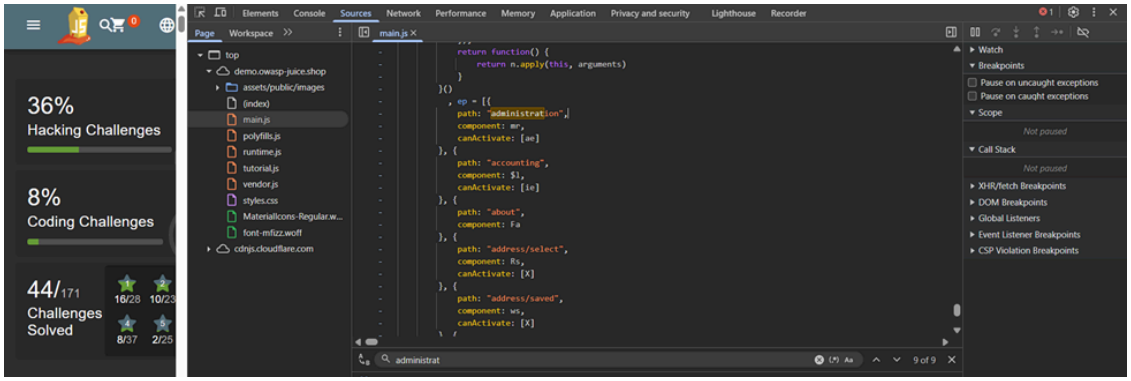
2.2 Broken Access Control – Unauthorized Access to Admin Panel

- **Severity**: High
- S**tep 1: Inspect the Web Page**
  - Open Developer Tools in the browser (F12).
  - Navigate to the

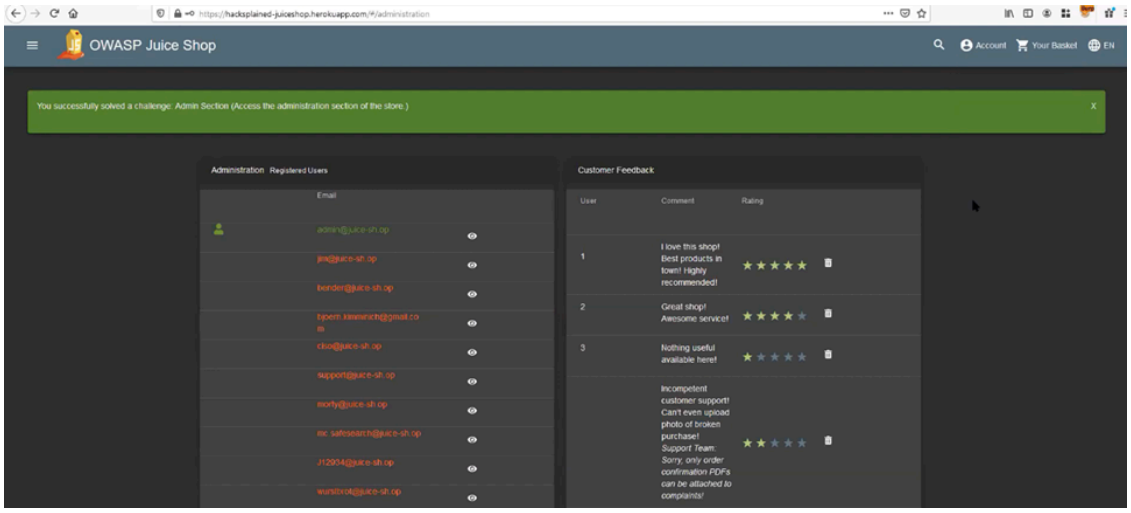
  Elements tab.
  - Noticed the presence of _ngcontent attributes.
  - Recognized the application uses Angular.



- **Step 2:Discover Hidden Routes**
  - Switched to the **Sources** tab in Developer Tools.
  - Opened and searched inside main.js under the debugger.
  - Searched for the keyword path.
  - Found the route path: 'administration'.



- **Step 3: Access the Admin Section**
  - Navigated to:https://demo.owasp-juice.shop/#/administration
  - The admin section loaded without any authentication prompt.



Impact

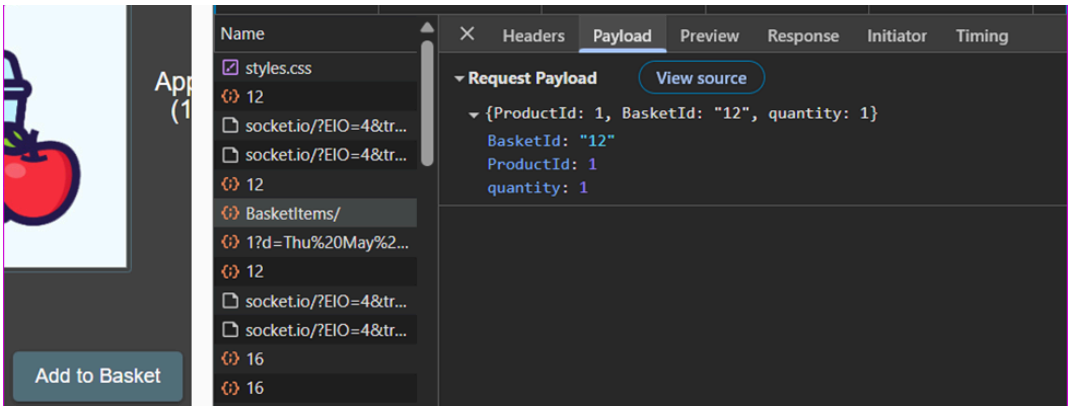
- Unauthorized access to admin features
- Possible exposure of sensitive configuration
- Risk of data manipulation or privilege abuse
- Violates principle of least privilege

Recommendations

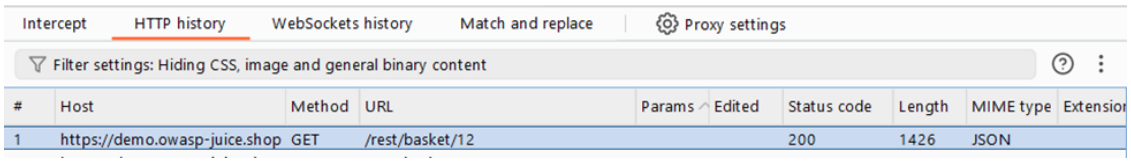
- Do not rely on obscurity (like hidden routes) to protect admin features.
- Implement proper server-side authorization checks for all restricted pages.
- Use route guards in Angular to enforce access control on the frontend.
- Log and alert on unauthorized access attempts to sensitive URLs

2.3 Insecure Direct Object References (IDOR)

- **Insecure Direct Object References (IDOR)**
- **Step 1: Send a POST request to access your own basket.**
  - **Accessed**: https://demo.owasp-juice.shop/api/BasketItems/
  - **Payload**:

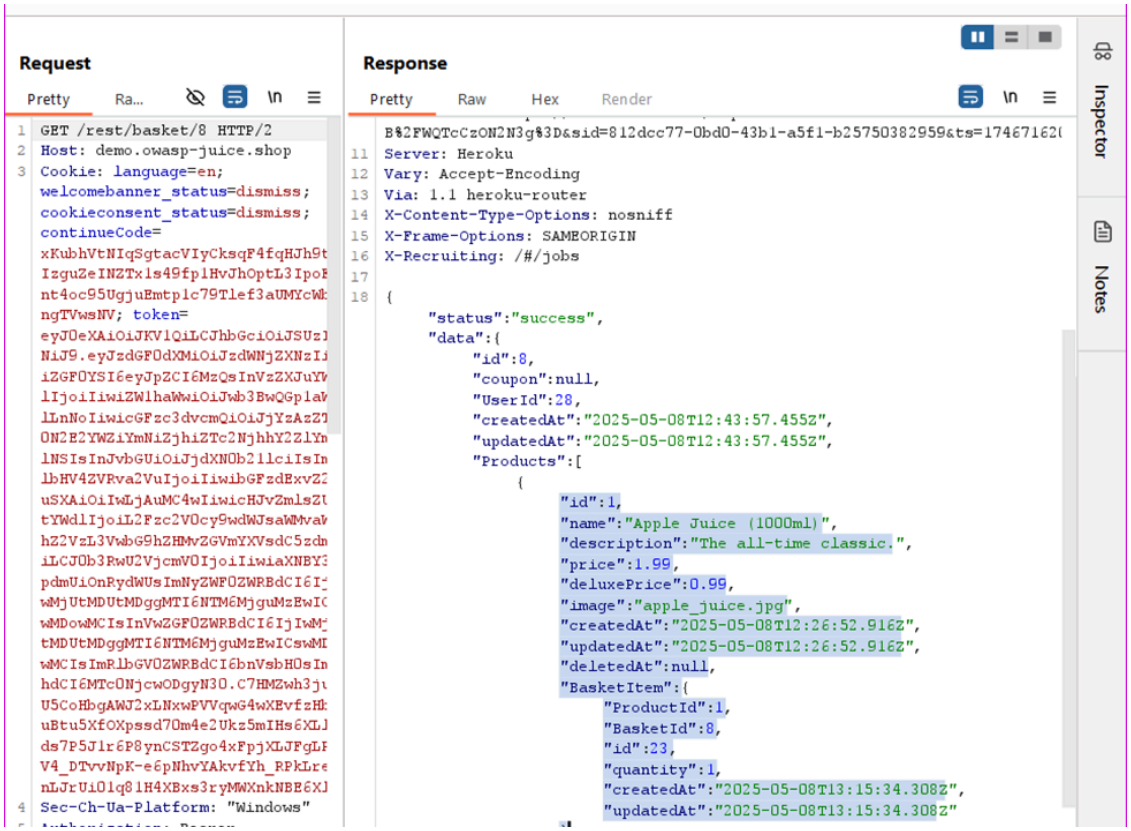


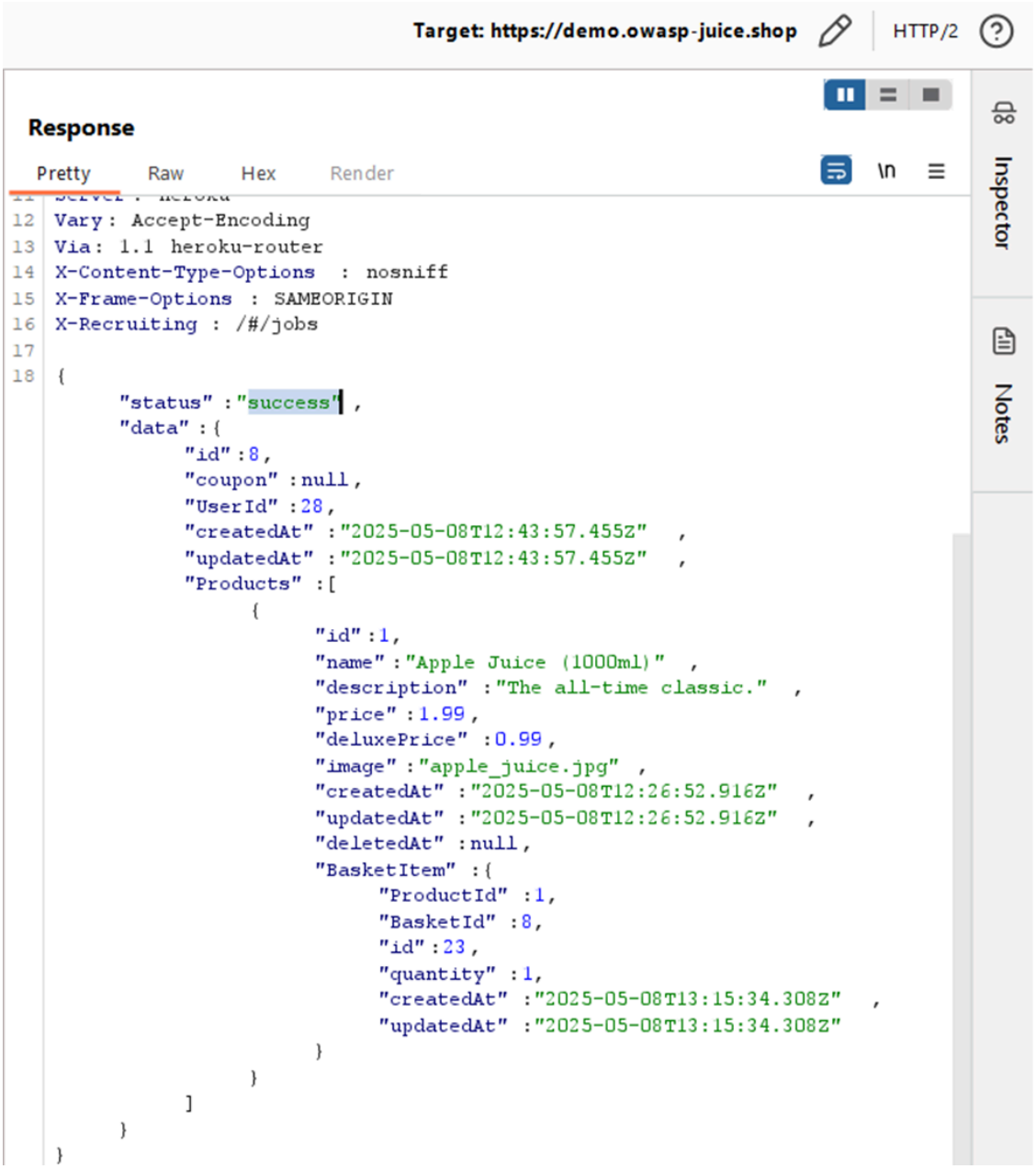
Result: The request is accepted, and you get a response showing your basket (BasketId = 12).



- **Step 2: Change BasketId and Observe Other User's BasketIdentifying BasketId 5:**
  - Modify the **BasketId** in the request to a different ID (e.g., 8) and send the request.

- The request is accepted, and the details of **BasketId = 8** (another user's basket) are returned.





- **Step 3: Modify Items in Another User's Basket**
  - Modify the contents of **BasketId = 8** by changing product details (e.g., adding more items).
  - The old payload:

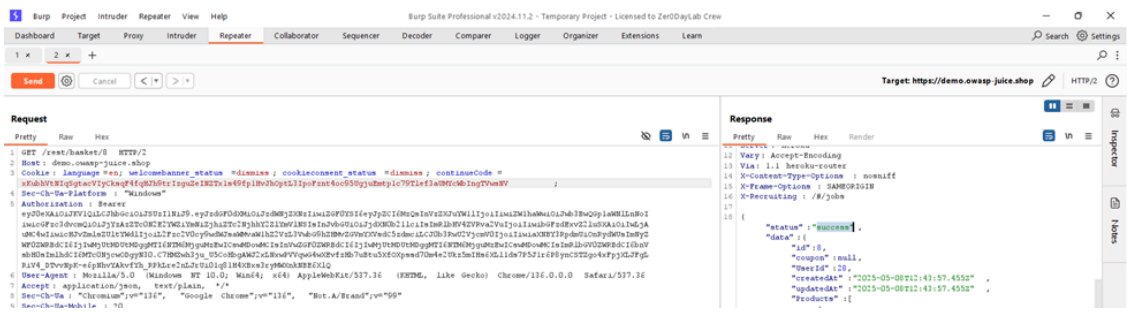
```
{
    "deleteCat": null,
    "BasketItem": {
        "ProductId": 1,
        "BasketId": 8,
        "id": 23,
        "quantity": 1,
        "createdAt": "2025-05-08T13:15:34.3082",
        "updatedAt": "2025-05-08T13:15:34.3082"
    }
}
```

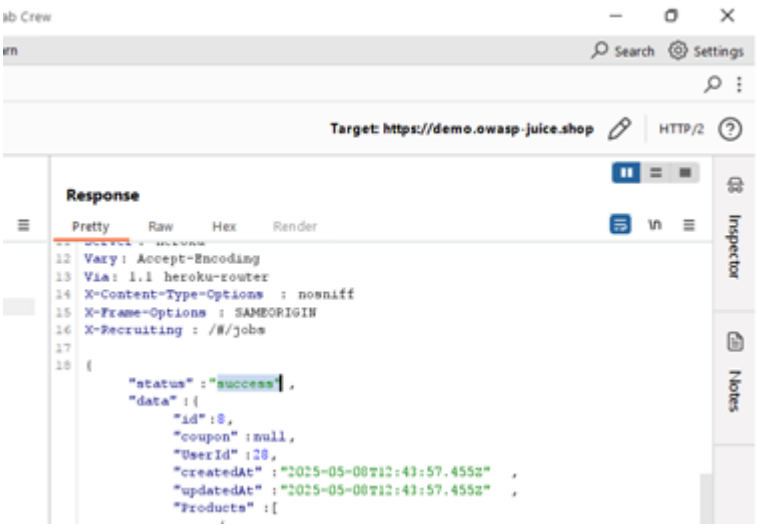

- The new payload:


```
{
    "deleteCat": null,
    "BasketItem": {
        "ProductId": 1,
        "BasketId": 8,
        "id": 23,
        "quantity": 10,
        "createdAt": "2025-05-08T13:15:34.3082",
        "updatedAt": "2025-05-08T13:15:34.3082"
    }
}
```


- **Result**: The basket contents are modified.

- **Step 4: Exploiting the Token Bypass**
  - Remove the **Authorization** token from the request and send the request again.
  - The request is still accepted, indicating that the application is not properly validating authentication.





Impact

- **Potential Damage**:
  - Unauthorized users can access or modify other users' baskets, potentially leading to information theft, unauthorized transactions, or data manipulation.
  - The ability to send requests without the necessary authentication token poses a significant security risk, as an attacker could exploit this weakness to bypass authentication entirely and manipulate or steal sensitive data.
- **Security Implications**:
  - This vulnerability indicates poor access control and could lead to unauthorized changes in the application’s data, which could be exploited by an attacker.
  - The lack of proper token validation during sensitive requests means that attackers can exploit this flaw to bypass authentication, potentially leading to unauthorized changes in critical data like user baskets, product inventories, or personal information.

Recommendations to Prevent Insecure Direct Object References (IDOR)

1. **Enforce Proper Access Control**

- Verify that the authenticated user has permission to access or modify the requested object.
- Ensure object-level access control checks are done on the server.

- Associate every resource (e.g. basket) with its owner and confirm that ownership before processing the request.

1. **Require Authentication for All Sensitive Operations**

- Reject any request that lacks a valid token.
- Do not allow actions like viewing or modifying user-related data without verifying the requester's identity.
- Avoid relying on client-side logic for enforcing authentication.

1. **Avoid Exposing Direct Object References**

- Do not include internal IDs (e.g. UserId, BasketId) in URLs or request bodies where not needed.
- Use indirect references such as UUIDs or securely mapped identifiers instead.

1. **Apply Least Privilege Principles**

- Limit what each user can access or modify based on their role and ownership.
- Do not allow users to escalate access by modifying IDs in requests.

1. **Monitor and Log Access to Sensitive Resources**

- Keep logs of resource access with user context (e.g. user ID, accessed BasketId)
- Detect and alert patterns that suggest IDOR attempts (e.g. access to multiple unrelated BasketIds)

1. **Regularly Test for IDOR Vulnerabilities**

- Perform security testing that includes manual ID manipulation.
- Include IDOR in your security checklists during development and QA.

2.4 Broken Authentication via Predictable OAuth Password Logic

- **Overview:**

  This is the most critical vulnerability discovered. The application uses OAuth for authentication, but instead of properly integrating with an OAuth provider like Google or GitHub, it uses a custom and insecure method to generate passwords based on the user's email.

  This allowed us to reverse-engineer the password generation logic and log in to a known admin account without the actual password.

- **Technical Details:**

  In the main.js file, we found the following logic:

  btoa(t.email.split('').reverse().join(''));

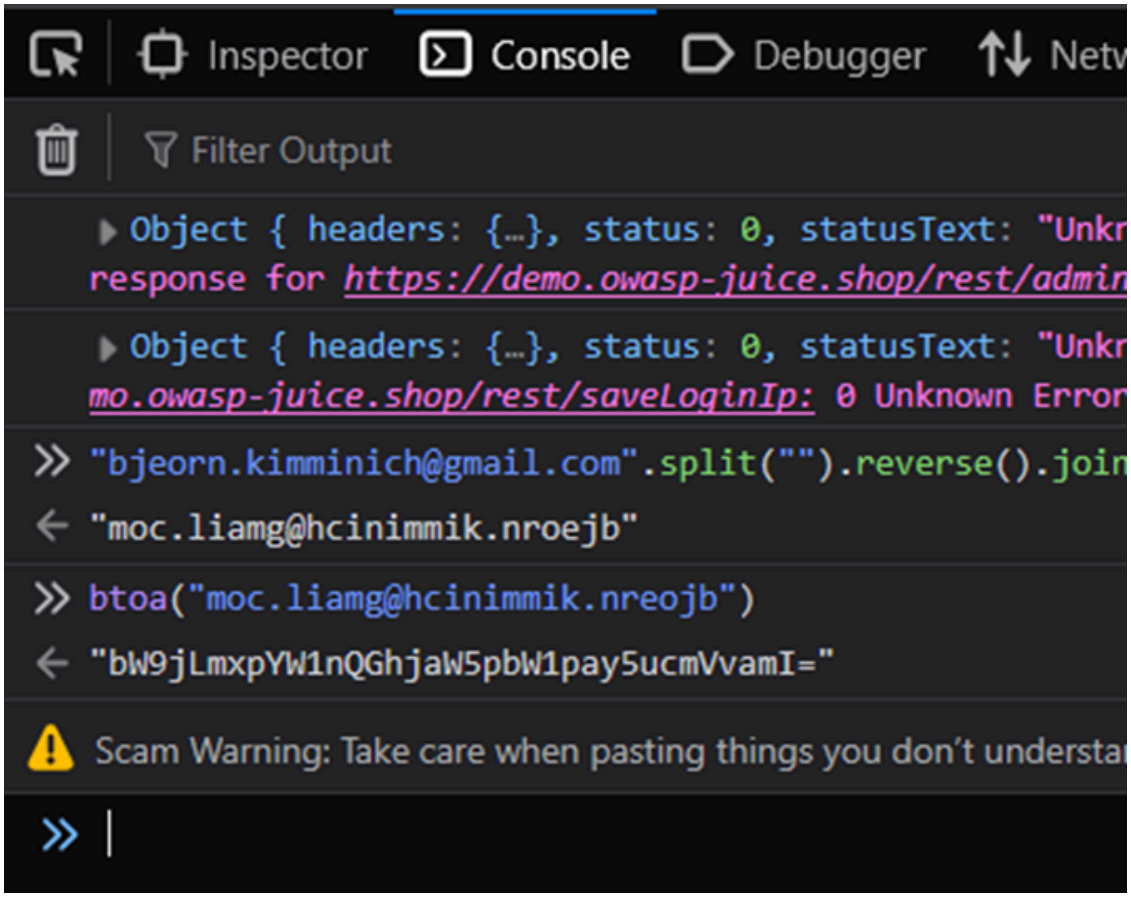
This means the application:

- Takes the user's email address.
- Reverses the characters.
- Encodes it using base64.
- Uses that as the password.

Knowing that the administrator's email was:

bjoern.kimminich@gmail.com

We applied the logic:



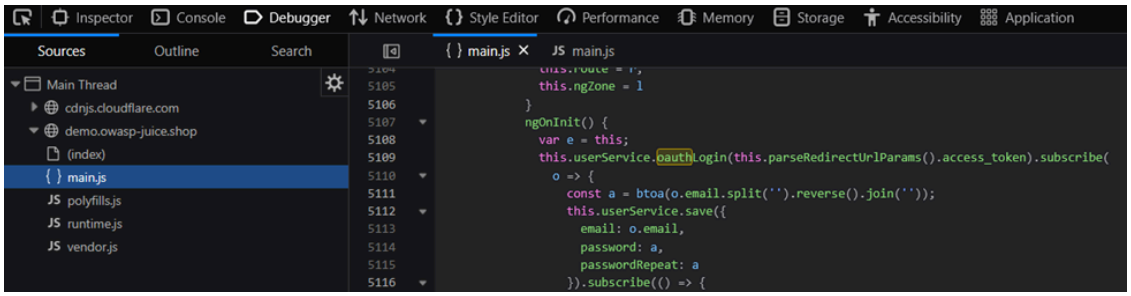
- Resulting password:

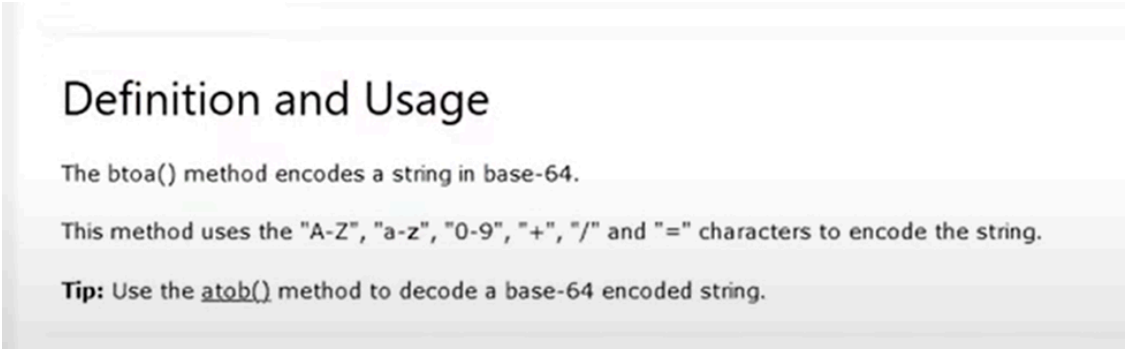
  bW9jLmxpYW1nQGhjaW5pbW1pay5ucmVvamI=

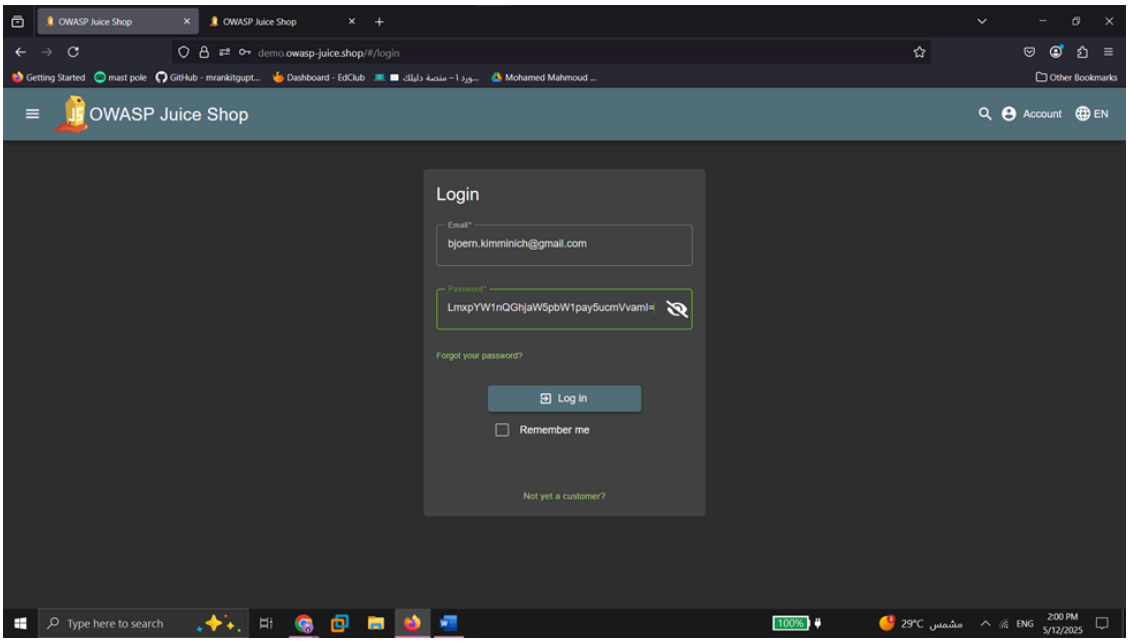
  We then logged in using:

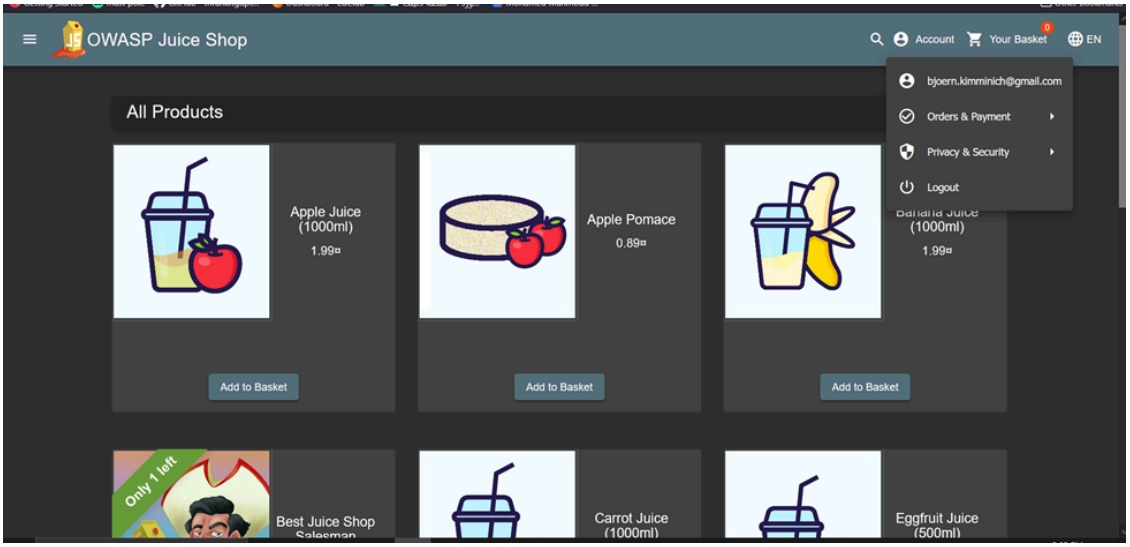
  - Email: bjoern.kimminich@gmail.com
  - Password: bW9jLmxpYW1nQGhjaW5pbW1pay5ucmVvamI=

  ...and were granted full administrator access.









- **Impact**

  This attack chain demonstrates full compromise of the application's most privileged account. The root cause is improper implementation of

authentication logic and lack of input validation.

- An attacker can:
  - Bypass login authentication entirely.
  - Discover and access hidden admin features.
  - Log in as any known user with just their email address.
- **Recommendations**
  - Implement proper server-side OAuth token validation.
  - Never store or generate passwords client-side.
  - Avoid exposing administrative routes in frontend code.
  - Sanitize all user inputs to prevent SQL Injection.
  - Protect sensitive functionality behind access control checks on the backend.

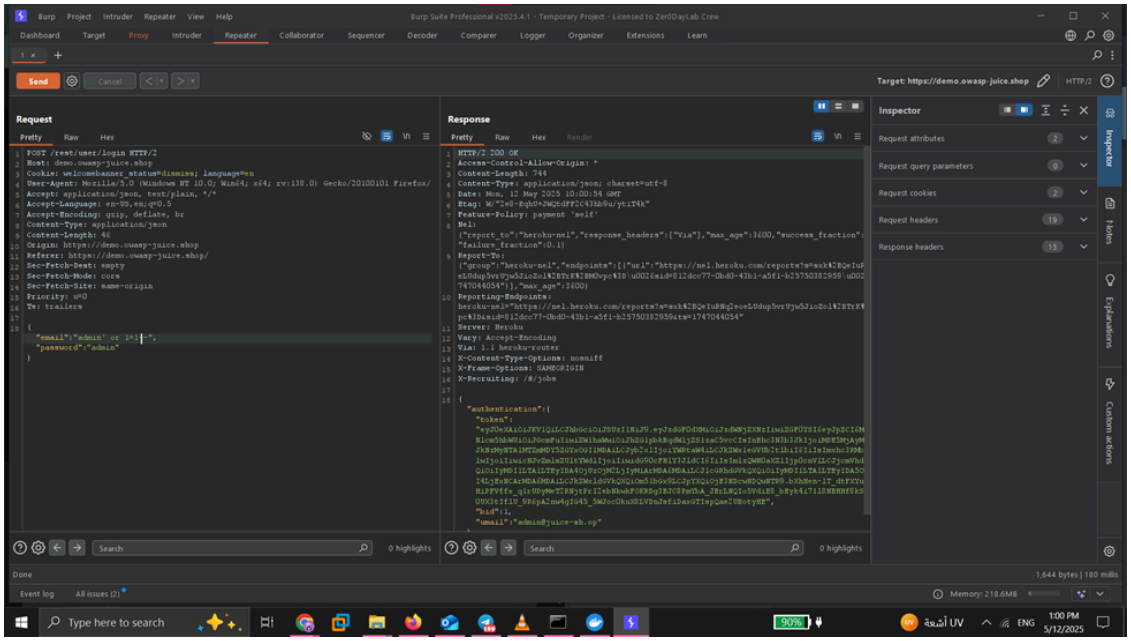
SQL Injection

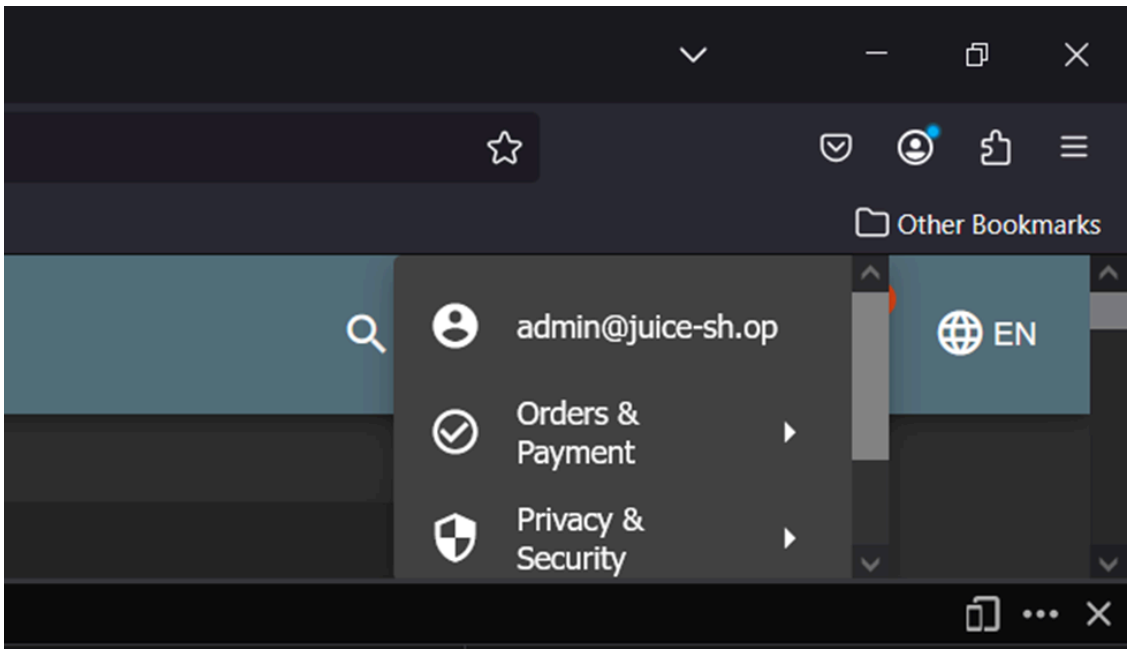
2.5 Authentication Bypass via SQL Injection

- Steps:
- Navigated to the login page.
- Entered the following payloads:
  - Email: `' OR 1=1--`
  - Password: (any value or blank)
- Result: Gained unauthorized access as the admin user.
- Navigated to /administration to view all registered users.

Impact:

- **Full administrative access** without valid credentials





2.6 Extracting Admin Credentials via JWT Token *(Maintaining Access)*

Steps:

- Used browser's Developer Tools and Cookie Editor to access the JWT token.

- Decoded the JWT payload using a Base64 decoder.
- Retrieved the password hash: `0192023a7bbd73250516f069df18b500`
- Used an online hash identifier and confirmed the hash type as **MD5**.
- Decrypted MD5 hash using public rainbow tables.
  - Password: `admin123`

Impact:

- Disclosure of sensitive credentials through insecure token storage.

2.7 SQL Injection to Bypass Login for Another User

Steps:

- Identified user email (`jim@juice-sh.op`) from product reviews.
- Attempted login with:
  - Email: `jim@juice-sh.op'--`
  - Password: (any value)
- Login was successful, bypassing password validation.
- Extracted JWT token, decoded it, and retrieved password: `ncc-1701`

Impact:

- Access to user accounts without valid credentials using SQL Injection.

2.8 Logging In with a Non-Existent Account via SQLi *(Maintaining Access)*

Steps:

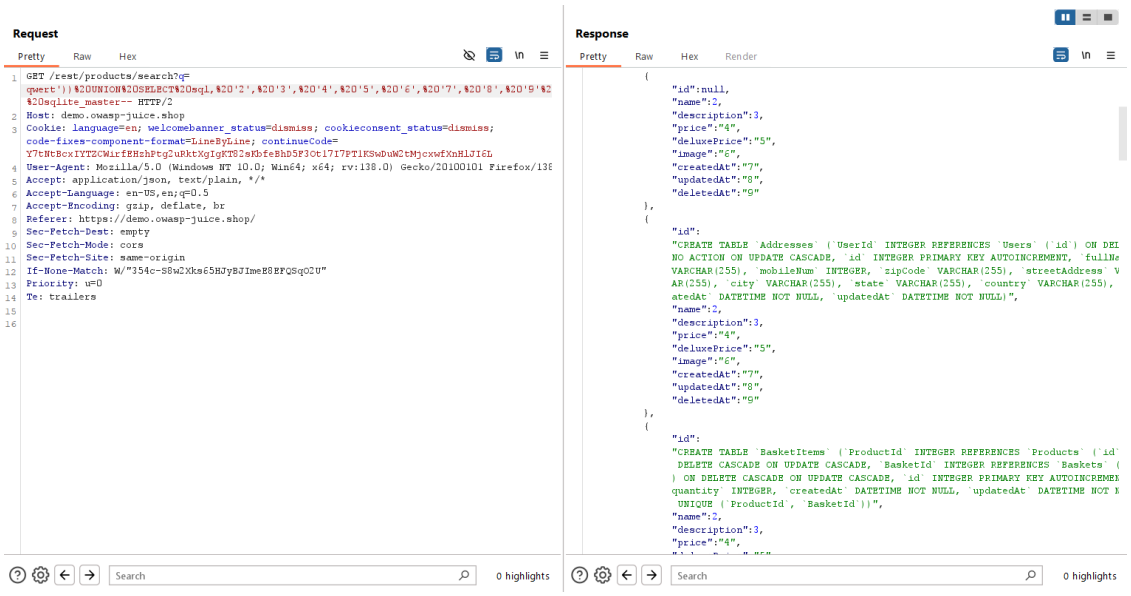
- Injected single quote `'` in email field and captured the request using Burp Suite.
- Error message indicated backend is using SQLite.
- Performed the following payload via search functionality:

```
banana')) UNION SELECT sql, '2', '3', '4', '5', '6', '7', '8', '9' FROM sqlite_master--
```

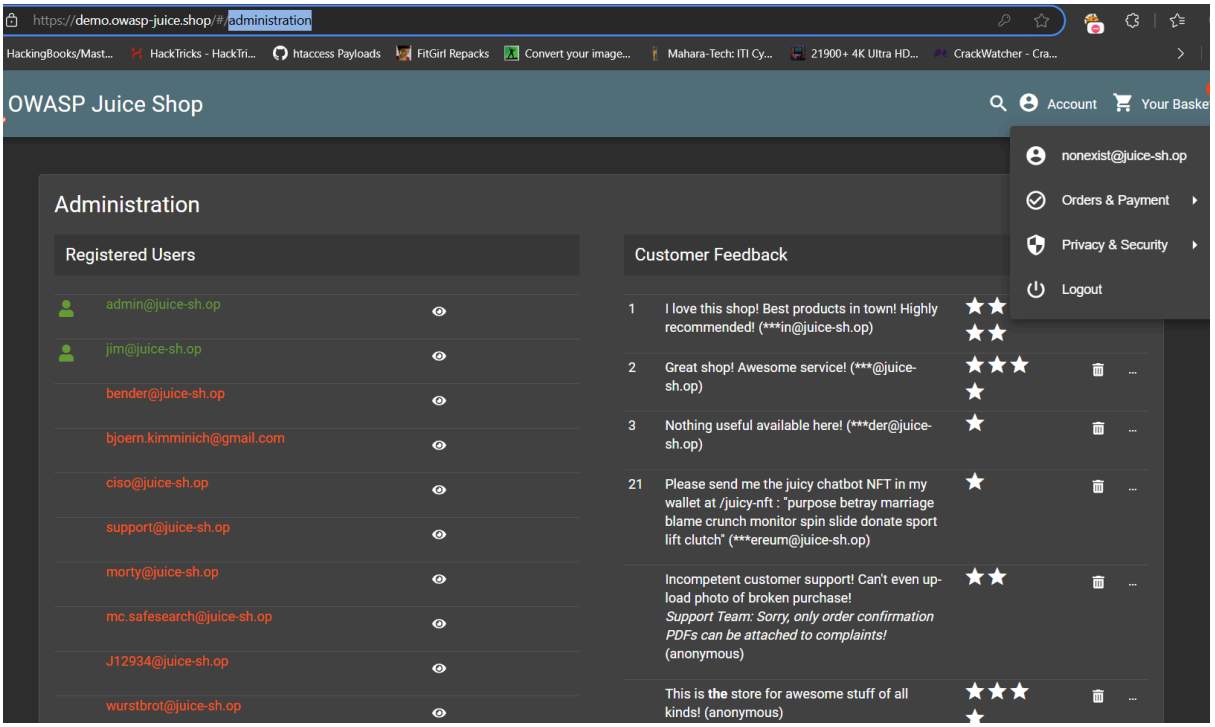
- Retrieved the database schema.
- Crafted and submitted the following UNION SELECT payload to create a fake user:

```
' UNION SELECT * FROM (SELECT 15 AS 'id', '' AS 'username', 'nonexist@juice-sh.op' AS 'email', '12345' AS 'password', 'admin' AS 'role', '123' AS 'deluxeToken', '1.2.3.4' AS 'lastLoginIp', '/assets/public/images/uploads/default.svg' AS 'profileImage', '' AS 'totpSecret', 1 AS 'isActive', '1999-08-16 14:14:41.644 +00:00' AS 'createdAt', '1999-08-16 14:33:41.930 +00:00' AS 'updatedAt', NULL AS 'deletedAt')--
```

- Successfully logged in as a fabricated account without registration.



- I made the non-exist account as admin and have all the authorizations:



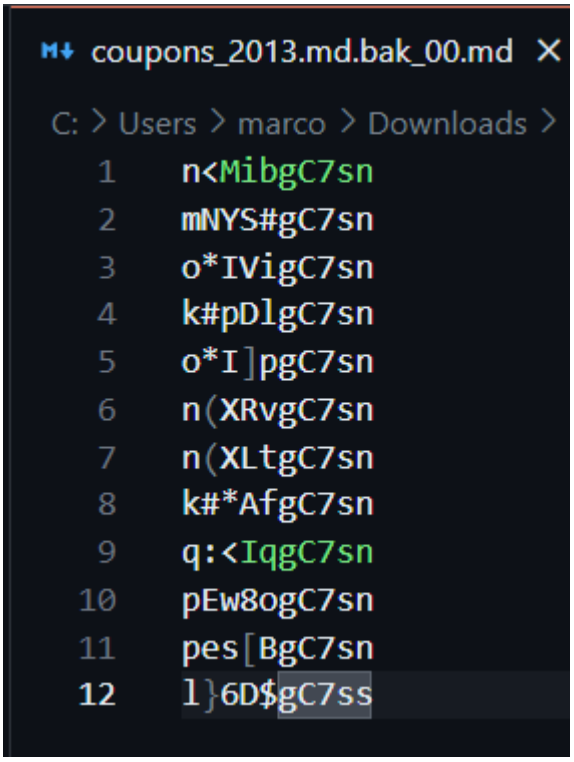
Impact:

- Full access with arbitrary user creation through SQL injection.

2.9 Discovery of Backup Files Using Gobuster

Steps:

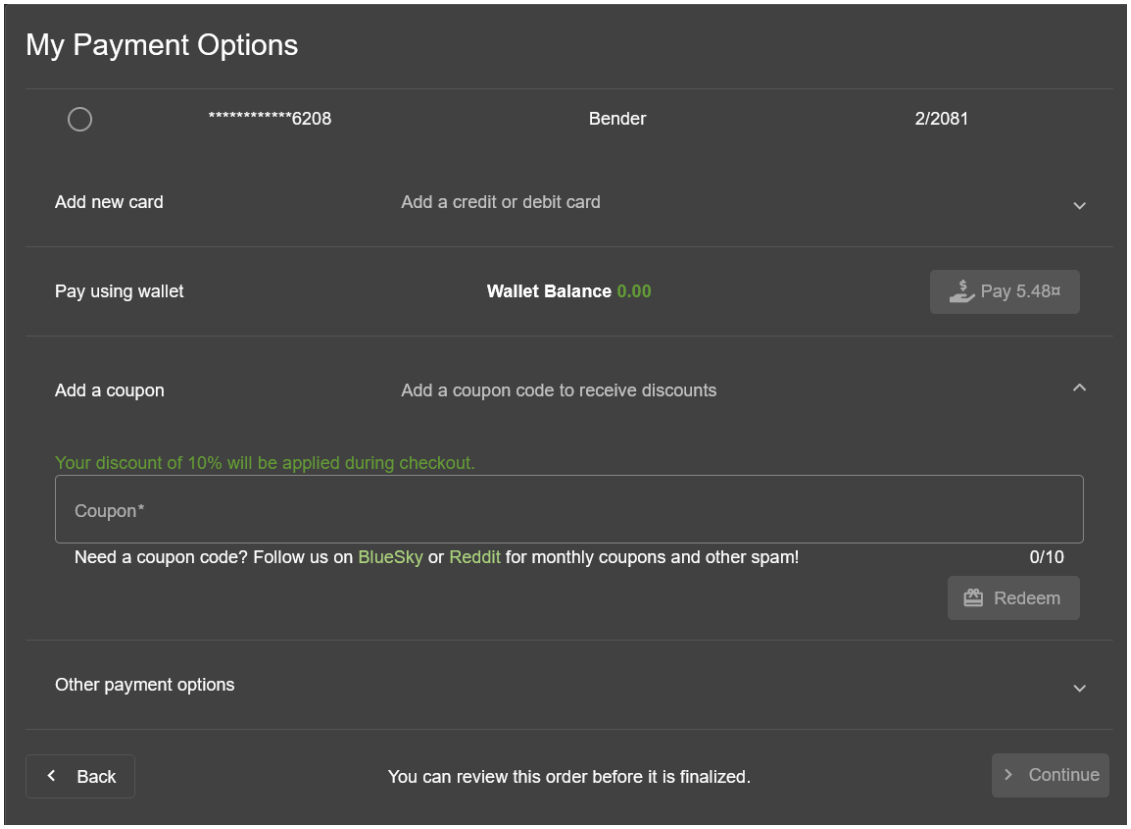
- Ran Gobuster on demo.owasp-juice.shop:
  - Discovered hidden directory: `/ftp`
  - Located backup file: `coupons_2013.md.bak` (initially forbidden)
- Bypassed access restriction by URL encoding:
  - Accessed file via `/ftp/coupons_2013.md%2500.bak`
- File contained historical coupon codes.



2.10 Analyzing Encryption Used for Coupons

Steps:

- Also downloaded package.json.bak from /ftp.
- Identified that Z85 encoding was used for coupon values.
- Example Coupons:
  - `pes[BgC7sn` → **NOV13-10** (Valid on 2013/10/11)
  - `k#*AfgC7sn` → **AUG13-10** (Valid on 2013/10/8)
- Encoded **MAY25-10** to **Z85** → `o*I]qh7ZKp`
- Successfully redeemed the generated coupon.



Impact:

- Unauthorized redemption of coupons through reverse-engineered encoding.

2.11 Cookies Missing HttpOnly Flags

1. **Summary**

During analysis of session management within the OWASP Juice Shop application, it was observed that cookies used to maintain authentication state are missing the HttpOnly attribute. This flag, when properly set, prevents client-side scripts from accessing the cookie via document.cookie, reducing the effectiveness of various client-side attacks, particularly Cross-Site Scripting (XSS).

1. **Technical Classification**

- **Vulnerability Type:** Cookies Missing Security Flags
- **CWE ID:** CWE-1004: Sensitive Cookie Without "HttpOnly" Flag
- **CVSS v3.0 Base Score:** 3.1 (Informational)
- **Attack Surface:** Session Cookies (e.g., token, session, or auth)

1. **Description**

Cookies issued by the server during authentication (e.g., JWT tokens or session identifiers) are not configured with the HttpOnly attribute. This exposes them to access via JavaScript running in the browser, which becomes a major security risk in the event of a successful XSS exploit.

Without the HttpOnly flag, any injected script can easily read the cookie value using document.cookie and exfiltrate it to an external attacker-controlled domain.

1. **Steps to Reproduce**
2. Navigate to the Juice Shop login page.
3. Authenticate using valid credentials.
4. Open the browser's developer tools → Application tab → Storage → Cookies.
5. Observe that the cookies (e.g., token) are:
    - Present
    - Not flagged as HttpOnly

Example output:

Set-Cookie: token=<JWT_VALUE>; Path=/; Secure; SameSite=Lax

No HttpOnly flag is present.

1. **Impact**

Risk: Moderate (under XSS conditions)

While the lack of HttpOnly alone does not constitute a direct attack vector, it significantly **amplifies the impact** of XSS vulnerabilities by making it trivial for malicious scripts to steal authentication cookies.

Scenarios:

- XSS leads to full session hijacking
- Stolen tokens used to impersonate users or admins
- Sensitive user data exposed due to weak session confidentiality

1. **Remediation**

Update the server-side cookie configuration to include the HttpOnly attribute on all sensitive cookies.

Example:

Set-Cookie: token=<JWT_VALUE>; Path=/; Secure; HttpOnly; SameSite=Lax

Recommendations:

- Apply HttpOnly to **all cookies** that do not require client-side script access
- Combine with Secure and SameSite flags for layered defense
- For session tokens, consider using **HttpOnly + Secure + SameSite=Strict**

1. **Additional Considerations**

- The HttpOnly flag does **not** prevent all XSS attacks, but it protects against **cookie theft**
- Additional safeguards such as **Content Security Policy (CSP)** and **input sanitization** must also be enforced
- Periodic cookie audits and secure defaults across all environments are advised

---

2.12 HTML Injection Leading to Stored Cross-Site Scripting (XSS)

1. **Summary**

A **stored XSS vulnerability** was identified in the OWASP Juice Shop application through the **Customer Feedback** submission feature. Malicious HTML and JavaScript code can be injected into the comment field, which is later rendered in the **Administration** interface without proper sanitization. This issue arises due to unsafe usage of document.innerHTML and improper handling of user-generated input.

1. **Technical Classification**

- **Vulnerability Type:** Stored Cross-Site Scripting (XSS)
- **CWE ID:** CWE-79: Improper Neutralization of Input During Web Page Generation
- **CVSS v3.0 Score:** 3.1 (Informational Risk)
- **Vector:** CVSS:3.0/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:N
- **Attack Surface:** Feedback submission → Admin Panel display (innerHTML sinks)

1. **Description**

The application fails to properly validate or encode user input before injecting it into the DOM using the innerHTML property. This makes it possible for an attacker to inject HTML tags — such as <a href=""> or potentially even <script> (under CSP bypass conditions) — into the administrator interface, which could escalate to full XSS under certain circumstances.

The vulnerability is stored, meaning the payload persists in the backend and is rendered for each subsequent admin page visit.

1. **Proof of Exploitation**
2. **Injected Payload via Feedback API:**

{

"UserId": 1,

"captchaId": 3,

"captcha": "6",

"comment": "fdsfasfd (**in@juice-sh.op)",

"rating": 4

}

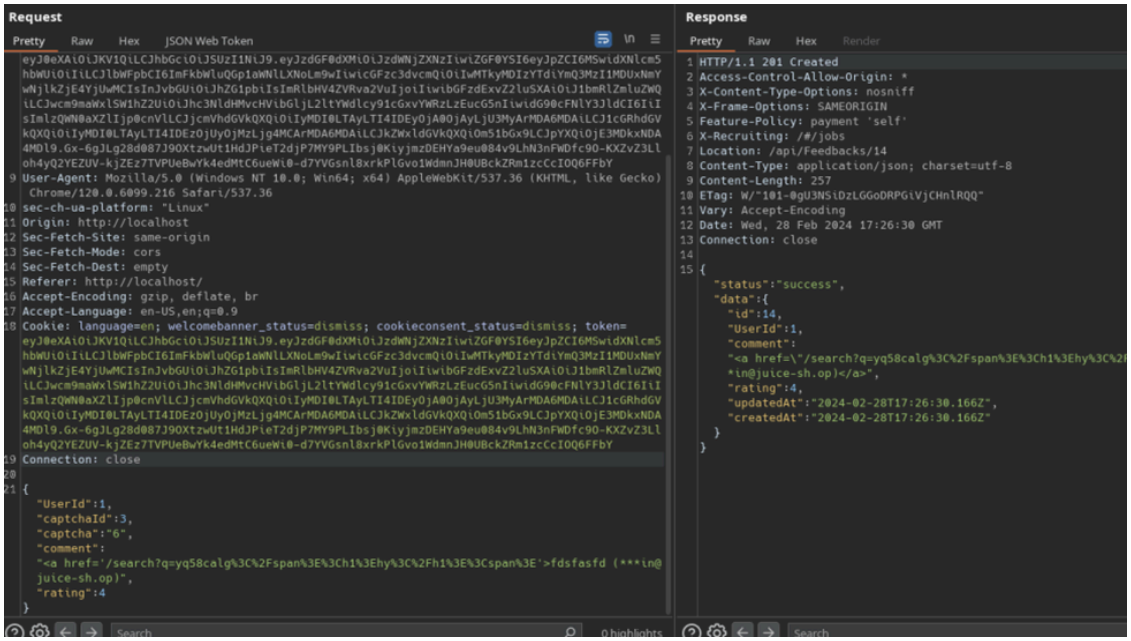
1. **Admin Panel Rendering:**

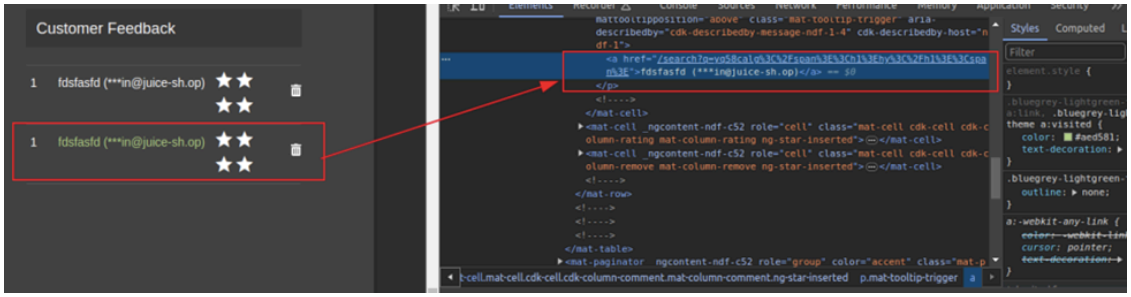
The feedback was rendered as HTML in the admin panel, proving that the comment was not sanitized or escaped, resulting in a clickable link inside the interface (see DOM inspection in screenshot).

1. **DOM Sink Confirmation:**

The payload appears inside a sink tracked as element.innerHTML:

e4s5picfl@e4s5picfl.localhost.net





1. **Risk Impact**

While the payload demonstrated was non-malicious, this vulnerability could be escalated to:

- **Phishing via spoofed links**
- **Session hijacking using malicious scripts (if CSP bypassed)**
- **Privilege escalation if chained with other admin functions**

1. **Remediation Strategy**

6.1 Input Validation & Output Encoding

- Apply strict **server-side validation** for feedback fields (reject HTML tags)
- Sanitize all user input using trusted libraries like DOMPurify (client-side) and validator.js (server-side)
- Escape output using secure templating systems that avoid raw HTML rendering

6.2 Secure DOM Updates

- Avoid innerHTML where possible; use textContent or safe DOM manipulation APIs

6.3 Enforce Content Security Policy (CSP)

- Deploy a restrictive CSP header such as:
- Content-Security-Policy: default-src 'self'; script-src 'none'; object-src 'none';
- Enable browser-based protection against inline scripts

6.4 Access Controls

- Implement **role-based access control (RBAC)** to limit exposure of feedback content to verified admins only
- Restrict who can access and trigger content rendering in /#/administration

6.5 Additional Measures

- Add **CSRF protection** to all form-based submissions
- Conduct **penetration testing** and **security code reviews** periodically
- Enhance **logging and monitoring** for unusual inputs and admin page behaviors

1. **Recommendations for Future Prevention**

- Adopt a **secure-by-default input handling policy**
- Educate developers on secure coding principles (e.g., never trust user input)
- Use modern frameworks and libraries with built-in protection against injection

- Regularly update dependencies and enforce secure headers at application and server levels

2.13 Cross-Site Request Forgery in Profile Management Endpoint

1. **Summary**

The OWASP Juice Shop application is vulnerable to **Cross-Site Request Forgery (CSRF)** on the /profile endpoint. This vulnerability allows an attacker to force authenticated users to submit unintended requests on their behalf, such as updating profile information without consent.

1. **Technical Classification**

- **Vulnerability Type**: Cross-Site Request Forgery
- **CWE ID**: CWE-352: Cross-Site Request Forgery
- **CVSS v3.0 Score**: 4.3 (Medium)
- **Affected Component**: /profile
- **Authentication Required**: Yes (user must be logged in)
- **User Interaction Required**: Yes (victim must visit a malicious site)

1. **Description**

The application does not verify the legitimacy of state-changing requests to the /profile endpoint. As a result, attackers can trick authenticated users into performing unintended actions—such as updating their username—by embedding malicious HTML on a third-party website.

This type of vulnerability can lead to unauthorized changes in user data and may serve as a stepping stone to more severe attacks, especially if combined with social engineering tactics or session fixation.

1. **Proof of Concept**

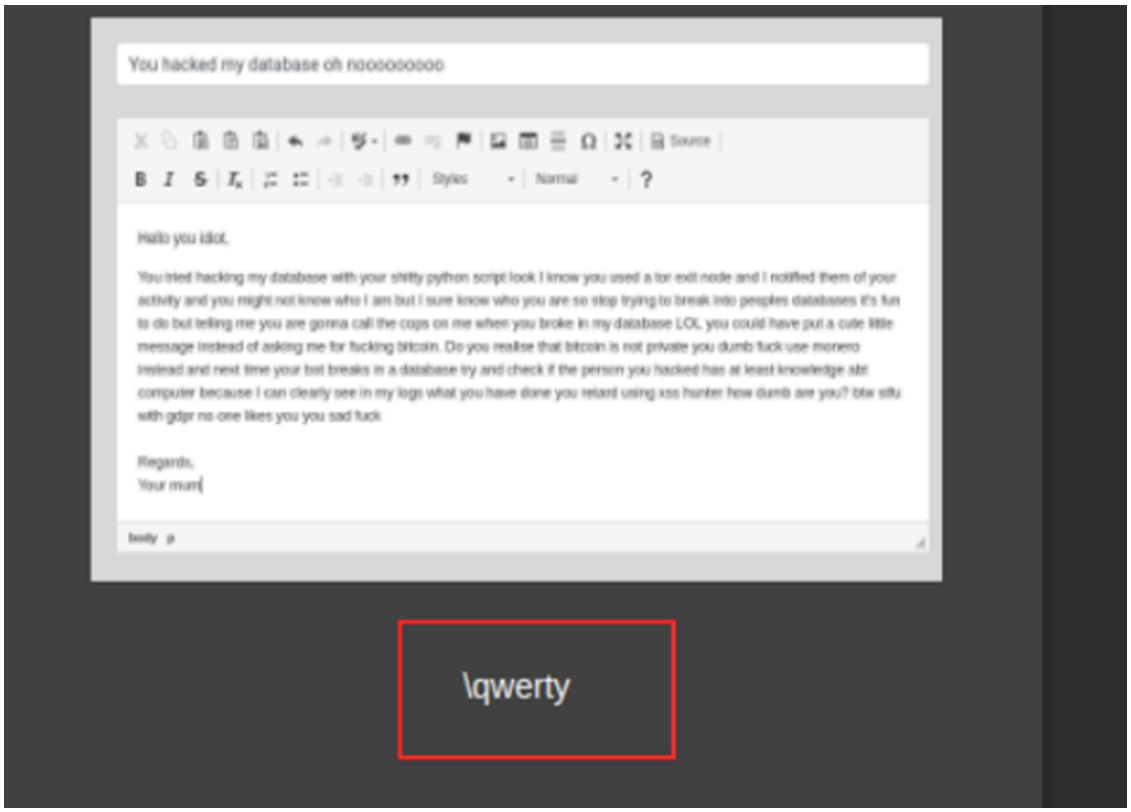
The following HTML page can be hosted by an attacker. When visited by a logged-in Juice Shop user, it silently changes their username to "qwerty" without their knowledge.

<html>

<body>

<form action="http://localhost/profile" method="POST">

```
<input type="hidden" name="username" value="qwerty" />
<input type="submit" value="Submit request" />
</form>
<script>
history.pushState('', '', '/');
document.forms[0].submit();
</script>
</body>
</html>
```



Reproduction Steps

1. Log in to Juice Shop as any valid user.
2. Host the above PoC HTML on an external web server.
3. In the same browser session, navigate to the hosted PoC.
4. Observe that the user's username changes without consent.
5. **Impact Analysis**

- **Confidentiality**: Not affected
- **Integrity**: Compromised — unauthorized changes to profile data
- **Availability**: Not affected

An attacker can exploit this vulnerability to manipulate user-facing data. While the immediate impact appears limited to profile tampering, it establishes a critical trust boundary violation in user action authentication.

1. **Remediation**

Remediation Reference:

Implement the following mitigation strategies immediately:

7.1 Backend CSRF Token Enforcement

Introduce CSRF protection on all state-changing forms using server-generated tokens validated upon submission.

Express.js Example (with csurf middleware):

```
const csrf = require('csurf');
const csrfProtection = csrf({ cookie: true });
app.use(csrfProtection);
// Inject csrfToken into the response
app.get('/profile', csrfProtection, function(req, res) {
res.render('profile', { csrfToken: req.csrfToken() });
});
```

In Form HTML:

```
<form method="POST" action="/profile">
<input type="hidden" name="_csrf" value="{{csrfToken}}">
<input type="text" name="username" />
<input type="submit" value="Update" />
</form>
```

7.2 Cookie Hardening

- Use SameSite=Lax or SameSite=Strict on all session cookies
- Mark cookies as Secure and HttpOnly where applicable

3- Maintaining Access

3.1 Admin Account Takeover via JWT Token Disclosure

Technique

- Extracted JWT token from browser cookies
- Decoded and retrieved the admin's MD5-hashed password
- Cracked the hash using public rainbow tables
- Reused admin credentials to log in repeatedly

Result

- Gained persistent access to the highest-privileged account
- No session expiration or token rotation detected

Persistence Potential

- Attacker can retain access unless the password is changed and token handling is secured

---

3.2 Creating a Backdoor Account via SQL Injection

Technique

- Performed SQL injection to enumerate the database schema
- Crafted a custom `UNION SELECT` payload to insert a new user with admin role
- Logged in with the fake account successfully

Result

- Gained long-term access as an admin without detection
- Account persisted in the database

Persistence Potential

- Attacker-controlled account remains unless manually removed
- Bypasses normal authentication and registration checks